Retries, Backoff & Jitter

how retries cause outages, and how to stop them

Watch the 4-min video

In one line: retry only transient failures on safe operations, wait with exponential backoff plus random jitter, cap attempts and total time with a deadline and a retry budget, and retry at one layer only.

1 Retries are powerful, and dangerous

Your checkout service calls the inventory service, which briefly returns **503**. One retry would have fixed it. But a thousand clients retrying instantly, all at the same moment, can turn a **two-second blip into a twenty-minute outage**. Retries add load at exactly the moment the dependency has the least capacity to spare.

2 Decide what to retry

Retry **transient** failures: connection errors, timeouts, **502, 503 and 504**, plus **429** after waiting the `Retry-After` delay the server asked for. Don't retry **400, 401, 403, 404 or 422**: the request itself is wrong, so it will just fail again.

And only retry operations that are **safe to repeat**. A timeout doesn't tell you whether the server did the work. Reads are safe; so are idempotent writes (set status = shipped) and writes protected by an **idempotency key**. Blindly retrying "charge card" can charge twice.



Figure 1. The two questions before any retry: is the failure likely to go away, and is it safe to do the operation again?

3 Exponential backoff

Don't retry immediately. Start with a **base delay** and **double it** on each attempt: 100 ms, then 200, 400, 800, up to a **cap** such as 10 seconds. That gives the struggling service room to recover, instead of hammering it harder.

$$backoff(n) = \min(cap,\ base \times 2^{n-1}) \quad \cdot \quad n = 1,\ 2,\ 3\ \ldots$$

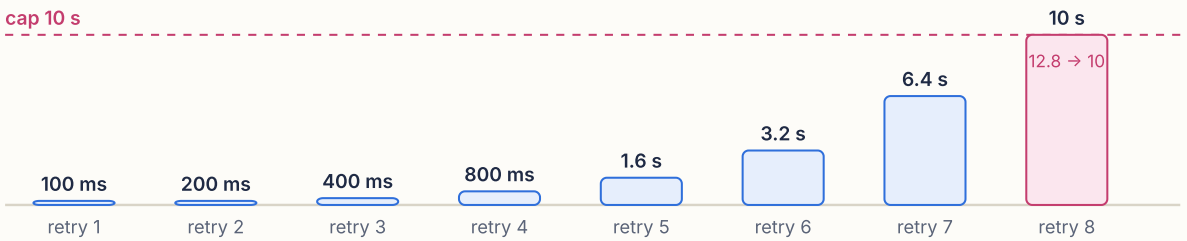


Figure 2. Each wait is twice the one before. The cap matters: without it, retry 11 would wait 100 ms × 2^{10} ≈ 102 s.

4 Jitter breaks up the waves

Backoff alone has a problem. If a thousand clients fail at the same moment, they all wait *exactly* 100 ms and retry together, in **synchronized waves**. Each wave can overload the service again. **Jitter** adds randomness so the retries spread out.

- **Full jitter**: sleep a random time between 0 and the backoff value. `sleep = random(0, backoff(n))`
- **Equal jitter**: keep half of the backoff, randomize the other half. `sleep = backoff(n)/2 + random(0, backoff(n)/2)`

AWS's analysis (the Architecture Blog post “Exponential Backoff and Jitter”) found **full jitter** spreads the load best, finishing the work with the fewest total calls.

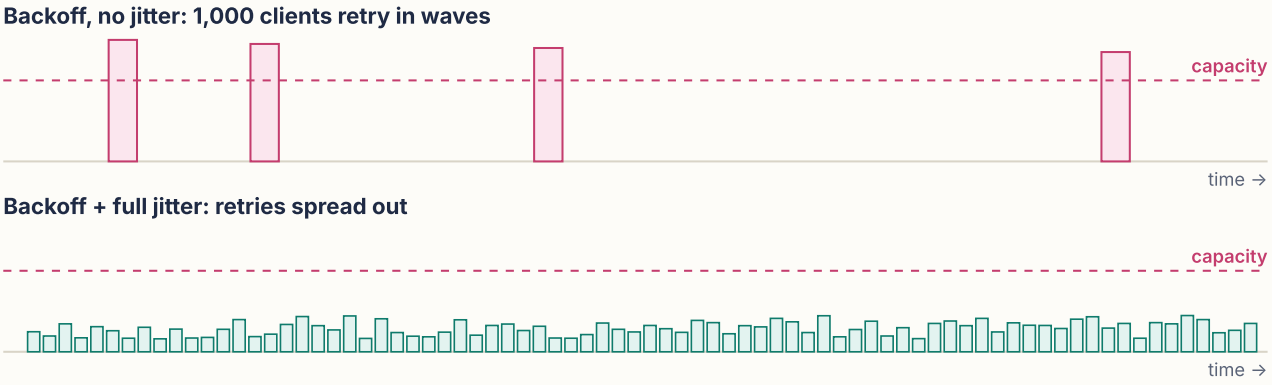


Figure 3. Top: identical delays line every client up, so each retry wave exceeds capacity. Bottom: random delays turn the waves into a steady trickle the service can absorb.

WORKED EXAMPLE: FULL JITTER DELAYS

Base 100 ms, cap 10 s, full jitter. Attempt 1 waits a random 0–100 ms, attempt 2 waits 0–200 ms, attempt 3 waits 0–400 ms. 1,000 clients that failed together now retry spread over a 400 ms window on the third try, about **2.5 per millisecond** instead of 1,000 in the same instant. By attempt 8 the window is 0–10 s (12.8 s capped to 10 s).

5 Set limits, and retry at one layer

- **Max attempts**, for example 3.
- **An overall deadline**, so retries never exceed what the caller is willing to wait (next lesson).
- **A retry budget**: allow retries only up to, say, **10% of normal traffic**, then fail fast. A budget stops retries from multiplying load when the system is weakest.

Watch out for **retry amplification**. If the gateway retries 3 times, the service retries 3 times and the database client retries 3 times, one user request can become **27 calls** to the database. Retry at **one layer**

only, usually the one closest to the user, and make inner layers fail fast. Pair retries with **circuit breakers**, which stop calling a dependency that is clearly down.

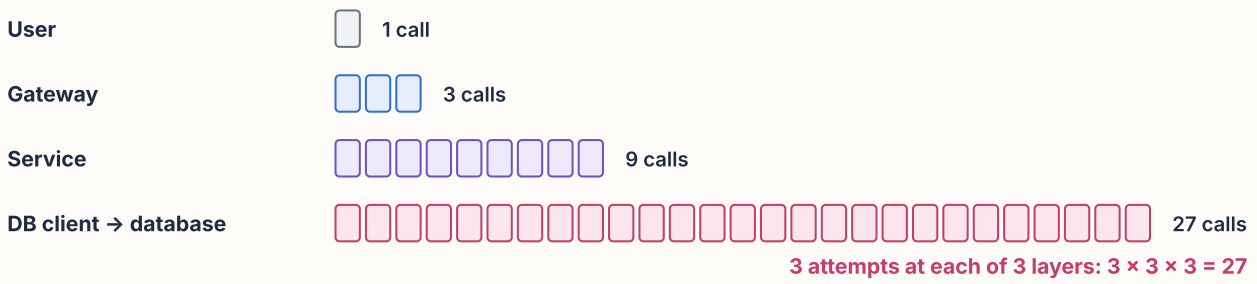


Figure 4. Retries multiply across layers. A database that is already slow receives 27 times the load from a single user click.

In Python: tenacity

```
from tenacity import (retry, stop_after_attempt, wait_random_exponential,
                      retry_if_exception_type, before_sleep_log)

@retry(stop=stop_after_attempt(3),
       wait=wait_random_exponential(multiplier=0.5, max=10),
       retry=retry_if_exception_type((Transient503, httpx.TimeoutException)),
       before_sleep=before_sleep_log(log, logging.WARNING))
def get_stock(sku): ...
```

Stop after three attempts, wait with random exponential backoff starting around half a second and capped at 10 seconds, retry only on the exceptions you consider transient, and **log every retry** so retry rates show up on your dashboards. A rising retry rate is often the first sign of trouble downstream.

COMMON MISTAKES

- Instant retries in a tight loop.
- Backoff with no jitter, so clients stay in lockstep.
- Retrying 4xx errors, or non-idempotent writes without an idempotency key.
- Retries at every layer (3 × 3 × 3 = 27).
- No attempt cap, no deadline, no budget.
- Ignoring `Retry-After` on 429 and 503.

6 Key terms

Transient failure: likely to succeed if tried again

Jitter: random part of the delay

Retry budget: retries capped at a share of traffic, e.g. 10%

Exponential backoff: delay doubles each attempt, up to a cap

Full jitter: random(0, backoff)

Retry amplification: retries multiply across layers

7 Interview questions

Q1. Which failures should you retry? BASIC

Transient ones: connection errors, timeouts, 502, 503, 504, and 429 after Retry-After. Not 400, 401, 403, 404 or 422. And only for operations that are safe to repeat: reads, idempotent writes or writes with an idempotency key.

Q2. Why add jitter to backoff? BASIC

Without it, clients that failed together retry together in synchronized waves that keep overloading the recovering service. Jitter spreads retries out randomly, smoothing the load and speeding recovery. Full jitter: random(0, backoff).

Q3. Write the backoff schedule for base 100 ms, cap 10 s. BASIC

100, 200, 400, 800 ms, 1.6, 3.2, 6.4 s, then 10 s (12.8 capped). With full jitter each is the upper bound of a random wait.

Q4. How can retries make an outage worse? SCENARIO

They multiply traffic exactly when a service is struggling: instant retries, no limits, and retries at several layers that amplify each other (3 layers × 3 attempts = 27 calls). Use backoff with jitter, attempt caps, retry budgets, one retry layer, and circuit breakers.

Q5. A payment call timed out. Should the client retry? SCENARIO

Only with an idempotency key. The timeout doesn't say whether the charge happened. With the same key, the server returns the first result instead of charging again; without one, check the payment status first.

Q6. Normal traffic is 2,000 RPS. What does a 10% retry budget mean? SCENARIO

At most about 200 retries per second across the fleet. Beyond that, calls fail fast instead of retrying, so a dependency outage adds at most 10% extra load instead of doubling or tripling it.

Q7. Gateway, service and DB client all retry 3 times. What do you change? TRICKY

Keep retries at one layer, usually the outermost or the one with the most context (often the edge or the calling service), and set the inner layers to fail fast with no retries. Add a deadline so retries stop when the user's budget is gone.

Q8. Retries vs circuit breaker: why use both? TRICKY

Retries handle brief, random failures. A circuit breaker notices a dependency is clearly down (e.g. 50% errors over 10 s) and stops calling it for a while, so retries don't keep hammering it. Retries fix blips; breakers protect during outages.

Next: SD-C-019 · API Timeouts & Deadline Propagation backendroadmap.com